Parallel-Constraint Model Predictive Control: Exploiting Parallel Computation for Improving Safety

Elias Fontanari[1], Gianni Lunardi[1], Matteo Saveriano[1] and Andrea Del Prete[1]

***Abstract*— Ensuring constraint satisfaction is a key requirement for safety-critical systems, which include most robotic platforms. For example, constraints can be used for modeling joint position/velocity/torque limits and collision avoidance. Constrained systems are often controlled using Model Predictive Control, because of its ability to naturally handle constraints, relying on numerical optimization. However, ensuring constraint satisfaction is challenging for nonlinear systems/constraints. A well-known tool to make controllers safe is the so-called *control-invariant set* (a.k.a. safe set). In our previous work, we have shown that safety can be improved by letting the safe-set constraint recede along the MPC horizon. In this paper, we push that idea further by exploiting parallel computation to improve safety. We solve several MPC problems at the same time, where each problem instantiates the safe-set constraint at a different time step along the horizon. Finally, the controller can select the best solution according to some user-defined criteria. We validated this idea through extensive simulations with a 3-joint robotic arm, showing that significant improvements can be achieved in terms of safety and performance, even using as little as 4 computational cores.**

I. Introduction

Guaranteeing safety is a fundamental requirement in almost all robotics applications. Safety is typically formulated via a set of equality/inequality constraints that the system should satisfy at all times. For instance, such constraints can model joint limits, actuation bounds, and collision avoidance. While formulating these constraints is typically straightforward, ensuring their persistent satisfaction is extremely challenging.

This is the case for both recent data-driven approaches, often relying on Reinforcement Learning (RL) algorithms, and for model-based control methods such as Model Predictive Control (MPC). Indeed, if the system dynamics is nonlinear, MPC methods in general cannot easily guarantee safety. The most common approach for guaranteeing safety relies on the knowledge of a so-called *safe set* (a.k.a. control-invariant set) [1], [2], or, equivalently, a Control Barrier Function (CBF) [3], [4].

Nonetheless, computing exact safe sets for nonlinear systems is generally unfeasible. Hence, professionals have relied on numerical methods that can generate approximations of such sets/functions [5]–[12]. Unfortunately, safety guarantees are compromised if the safe set is not exact. A more in-depth discussion of MPC methods that can guarantee safety is postponed to Section II.

An alternative to MPC is *Model Predictive Shielding* (MPS) [13], [14]. This approach assumes the knowledge of a backup policy that can drive the state to an invariant set. Contrary to the invariant set required by MPC methods, this invariant set can be potentially very small, without negatively affecting the basin of attraction of the controller. However, finding a backup policy and the associated (small) invariant set has a similar complexity to finding a (large) safe set.

Despite the abundant literature on safe control methods, little work has focused on exploiting parallel computing for improving safety. MPC optimization problems can be solved exploiting parallel computation, either on CPU, GPU, or FPGA [15]–[17], but optimization algorithms are hard to parallelize and therefore only limited speed-ups can be achieved. Moreover, solving the MPC faster does not directly improve safety, as it simply reduces the time delay due to the computation.

Sampling-based MPC methods [18]–[20] are natural candidates for parallel computation, given the simple parallelization of their algorithms. However, they do not address directly the safety aspect. The only exception is Statistical MPS, which relies on forward simulating the backup policy considering different instances of the system noise [14]. While the authors did not discuss this aspect, it is clear that such operation can easily exploit parallel computation. However, this only makes sense when the system is subject to noise and a statistical certification is acceptable.

Our main contribution is a novel MPC scheme that exploits parallel computation to directly improve safety. This is fundamentally different from previous work exploiting parallel computation for MPC, because we do not merely try to solve the problem faster, but instead try to achieve a *better* solution (in terms of safety), in the same amount of time. Our method can be seen as an extension of our previous work on *Receding-Constraint MPC* [21] (summarized in Section III), where we showed that by making the safe-set constraint recede along the horizon we could achieve superior theoretical and practical safety than classic terminal-constraint approaches. With *Parallel-Constraint MPC* (described in Section IV) we push this idea further. Rather than solving a single MPC problem, we solve several MPC problems, each one constraining the state inside the safe set at a different time step along the horizon. This enhances our chances of getting a feasible solution, therefore improving safety. Moreover, among all solved problems, we can choose the one that satisfies the safe-set constraint the furthest along the horizon,

[1]The authors are with the Industrial Engineering Department, University of Trento, Via Sommarive 11, 38123, Trento, Italy. {name.surname}@unitn.it

Financed by the European Union - Next Generation EU, Mission 4 Component 2 - CUP E53D23001130006 (STARLIT), CUP E53D23001020001 (ARIEL), and project INVERSE (Grant Agreement n. 101136067).

therefore enlarging our safety window. Our simulation results with a 3-degree-of-freedom robot manipulator, reported in Section V, show that *Parallel-Constraint* MPC can significantly outperform *Receding-Constraint* MPC, completing up to 21% more tasks while failing up to 6% less tasks. Significant improvements were also achieved when using a limited number of computational units (between 4 and 16).

II. Preliminaries

This section quickly reviews a few relevant concepts from the literature: backward-reachable sets, control-invariant sets, and Recursive Feasibility in MPC.

A. Notation

- $\mathbb{N}$ denotes the set of natural numbers;
- $\{x_i\}_0^N$ and X denote a discrete-time trajectory given by the sequence $(x_0, \dots, x_N)$;
- $x_{i|k}$ denotes the state at time step $k+i$ predicted when solving the MPC problem at time step k;

B. Problem statement

Let us consider a discrete-time dynamical system with state and control constraints:

$$x_{i+1} = f(x_i, u_i), \qquad x \in \mathcal{X}, \qquad u \in \mathcal{U}. \tag{1}$$

The objective is to design a controller that ensures *safety* (i.e., constraint satisfaction), while minimizing a user-defined cost function. We define $\mathcal{S}$ as the subset of $\mathcal{X}$ containing all the equilibrium states:

$$\mathcal{S} = \{x \in \mathcal{X} \mid \exists\, u \in \mathcal{U} : x = f(x, u)\}. \tag{2}$$

To ensure safety, we will rely on the *M-Step Backward-Reachable Set* [1] of $\mathcal{S}$, which we denote as $\mathcal{V}_M$. Mathematically, it is defined as the subset of $\mathcal{X}$ starting from which it is possible to reach $\mathcal{S}$ in M steps:

$$\mathcal{V}_M \triangleq \{x_0 \in \mathcal{X} \mid \exists \{u_i\}_0^{M-1} : x_M \in \mathcal{S}, x_i \in \mathcal{X}, \\ u_i \in \mathcal{U}, \forall\, i = 0, \dots, M-1\}. \tag{3}$$

Being a backward reachable set of equilibrium states, the set $\mathcal{V}_M$ is control-invariant [1]. Namely, starting inside $\mathcal{V}_M$, we can remain inside $\mathcal{V}_M$ forever. Knowledge of the set $\mathcal{V}_M$ would make it easy to design a safe controller. However, we cannot assume to know $\mathcal{V}_M$ in general, because its computation can be extremely complex. We rely instead on the following, more realistic, assumption.

Assumption 1. *We know a conservative approximation of the set $\mathcal{V}_M$:*

$$\hat{\mathcal{V}}_M \subseteq \mathcal{V}_M \tag{4}$$

Note that $\hat{\mathcal{V}}_M$ need not be control invariant in general.

As discussed above, different methods exist to compute numerical approximations of $\mathcal{V}_M$. We have chosen to use a slightly modified version of the Viability-Boundary Optimal Control (VBOC) method [12]. The resulting approximation of $\mathcal{V}_M$ can be easily made conservative by introducing a user-defined safety margin, which we explain in more detail in Section V. In the following, we summarize standard MPC approaches to exploit $\hat{\mathcal{V}}_M$ for achieving safety.

C. Model Predictive Control and Recursive Feasibility

Let us consider the following MPC problem:

$$\begin{aligned}
\underset{X,U}{\text{minimize}} \quad & \sum_{i=0}^{N-1} \ell_i(x_i, u_i) + \ell_N(x_N) && \text{(5a)}\\
\text{subject to} \quad & x_0 = x_{init} && \text{(5b)}\\
& x_{i+1} = f(x_i, u_i) \qquad i = 0 \dots N-1 && \text{(5c)}\\
& x_i \in \mathcal{X}, u_i \in \mathcal{U} \qquad i = 0 \dots N-1 && \text{(5d)}\\
& x_N \in \mathcal{X}_N && \text{(5e)}
\end{aligned}$$

where $\ell(\cdot)/\ell_N(\cdot)$ is the running/terminal cost, x_{init} is the current state, and $\mathcal{X}_N \subseteq \mathcal{X}$ is the terminal set [22].

Even if MPC is probably the most commonly used approach for controlling constrained systems, ensuring constraint satisfaction remains difficult when the dynamics or the constraints are nonlinear. A well-known approach to ensure safety is to rely on Recursive Feasibility (RF). RF guarantees that, if an MPC problem is feasible at the first loop, it remains feasible forever.

RF can be guaranteed by choosing the terminal set $\mathcal{X}_N$ to be a *control-invariant* set. While theoretically sound, the practical problem of this approach is that control-invariant sets are in general extremely challenging to compute for nonlinear systems/constraints.

Since in general we only know $\hat{\mathcal{V}}_M$, which may not be control invariant, using it as terminal set does not ensure RF. In other words, the MPC problem could become unfeasible. To deal with unfeasibility, one can relax the terminal constraint using a slack variable, which is heavily penalized in the cost function [23], [24]. This can temporarily recover feasibility, in the hope that perhaps the terminal constraint be satisfied again in the future. However, with this approach we cannot ensure *safety*, nor RF. Indeed, the slack variable allows the state to leave $\hat{\mathcal{V}}_M$, which eventually can lead to leaving $\mathcal{X}$.

III. Receding-Constraint MPC with Safe Abort

In our previous work [21], we have introduced a novel MPC scheme that relies on two components: a Receding-Constraint MPC formulation and a safe task-abortion strategy. We summarize them in the following, in a slightly generalized form.

A. Receding-Constraint MPC

The key idea of this MPC scheme is to adapt online the time step at which the state is constrained in $\hat{\mathcal{V}}_M$, making it recede along the horizon. If at MPC loop $k-1$ we had $x_{r|k-1} \in \hat{\mathcal{V}}_M$, at loop k we can be sure that it is possible to have $x_{r-1|k} \in \hat{\mathcal{V}}_M$ (assuming no disturbances and modeling errors). Thus the constraint can be established in a hard way. This guarantees safety for r loops, where this *receding constraint* slides backward along the horizon. However, once it reaches time step 0, we can no longer rely on it to ensure safety. Therefore, we maintain also a

soft constraint to encourage the terminal state to be in $\hat{\mathcal{V}}_M$, leading to this MPC formulation:

$$\begin{aligned} \underset{X,U,s}{\text{minimize}} \quad & \sum_{i=0}^{N-1} \ell_i(x_i, u_i) + \ell_N(x_N) + w_s|s| \\ \text{subject to} \quad & \text{(5b)}, \text{(5c)}, \text{(5d)} \\ & x_r \in \hat{\mathcal{V}}_M,\, x_N \in \hat{\mathcal{V}}_M \oplus s. \end{aligned} \tag{6}$$

After solving the MPC at loop $k-1$, we check whether $x_{N|k-1} \in \hat{\mathcal{V}}_M$; if so, at loop k we move the receding constraint forward on $x_{N-1|k}$. Actually, we can even check whether $x_i \in \hat{\mathcal{V}}_M$, for any $i > r$, and if that is the case we can set $r = i - 1$ at the next loop. Otherwise, we let r decrease. If we reach $r = 1 + L$ (with L being a small non-negative user-defined integer, whose role is clarified in the following) then we trigger the safe task abort procedure.

B. Safe Task Abortion

Our key idea to ensure safety relies on Assumption 1 and on the following two assumptions.

Assumption 2. *We have access to two computational units: unit A and unit B.*

Assumption 3. *We can solve the following Optimal Control Problem (OCP) for any $x_{init} \in \hat{\mathcal{V}}_M$, in $L+1$ time steps (with $0 \leq L \leq N-1$):*

$$\begin{aligned} \underset{\{x_i\}_0^M, \{u_i\}_0^{M-1}}{\text{minimize}} \quad & \sum_{i=0}^{M-1} \ell_i(x_i, u_i) + \ell_M(x_M) \\ \text{subject to} \quad & \text{(5b)}, \text{(5c)}, \text{(5d)},\, x_M = x_{M-1} \end{aligned} \tag{7}$$

The choice of the cost function is irrelevant.

OCP (7) finds a trajectory reaching an equilibrium state from x_{init}. Suppose we trigger the safe abort procedure at MPC loop k. The procedure consists of these steps:

1) unit A uses the MPC solution computed at loop $k-1$ to reach the safe state $x_{L+1|k-1} \in \hat{\mathcal{V}}_M$;
2) in parallel, unit B solves OCP (7), using $x_{L+1|k-1}$ as initial state;
3) after reaching $x_{L+1|k-1}$, we follow the solution of OCP (7) to reach an equilibrium state.

The hyper-parameter L should be set by the user as small as possible, while satisfying Assumption 3.

C. Theoretical Properties

In [21], we have shown that the Receding-Constraint MPC scheme, coupled with the safe abort procedure, enjoys two interesting theoretical properties.

The safe abort procedure ensures *safety* as long as Assumptions 1, 2 and 3 are satisfied. Moreover, Receding-Constraint MPC ensures *recursive feasibility* under the assumption that $\hat{\mathcal{V}}_M$ is $(N-L)$-step control invariant. This means that, from any state inside $\hat{\mathcal{V}}_M$, it must be possible to reach a state in $\hat{\mathcal{V}}_M$ in at most $N-L$ time steps. This property is clearly weaker than the classic control invariance that is required by terminal constraint approaches.

Algorithm 1 Parallel-Constraint MPC with Task Abortion

Require: Number of time steps T, Initial state x_0, OCP (9), Safe-abort OCP (7), Safe-abort trigger time L

```
r ← N                                          ▷ Safe horizon
for t = 0 → T − 1 do
    parallel-for p ∈ [1, N] do
        X_p, U_p ← OCP(p, x_t)
        r_p ← NaN
        for k = r → N do                       ▷ Search last state in V̂_M
            if X_p[k] ∈ V̂_M then
                r_p ← k
    p ← ArgMax({r_p}_1^N)                      ▷ Find best solution
    if p is NaN then                           ▷ Use previous solution
        r ← r − 1
        X*[: N − 1], U*[: N − 2] ← X*[1 :], U*[1 :]
    else
        r ← r_p − 1
        X*, U* ← X_p, U_p
    if r = L then                              ▷ Trigger Safe Abort
        TriggerSafeAbort(x*_L)                 ▷ Solve (7) in Unit B and then follow the computed trajectory
    x_{t+1} ← f(x_t, u*_0)                     ▷ Simulate dynamics
```

IV. Parallel-Constraint Model Predictive Control

Instead of using a single state along the horizon to ensure safety, we could try to reach the safe set at different time steps exploiting parallel computation. As long as at least one state $x_p \in \hat{\mathcal{V}}_M$ (with $1 \leq p \leq N$), we can ensure constraint satisfaction because $x_1 \in \mathcal{V}_{M+p-1}$. This is because from x_1 we can reach x_p in $p-1$ steps. Ideally, we would like to include the following constraint in our problem formulation:

$$(x_1 \in \hat{\mathcal{V}}_M) \vee (x_2 \in \hat{\mathcal{V}}_M) \vee \ldots \vee (x_N \in \hat{\mathcal{V}}_M). \tag{8}$$

However, this type of constraint (OR constraints) is extremely hard to deal with for numerical solvers.

If we have access to N computational units, we can solve N problems in parallel, each constraining the state inside $\hat{\mathcal{V}}_M$ at a different time step $p \in [1, N]$:

$$\begin{aligned} \underset{X,U}{\text{minimize}} \quad & \sum_{i=0}^{N-1} \ell_i(x_i, u_i) + \ell_N(x_N) \\ \text{subject to} \quad & \text{(5b)}, \text{(5c)}, \text{(5d)} \\ & x_p \in \hat{\mathcal{V}}_M \end{aligned} \tag{9}$$

Then, we can use the solution of one of the problems that have been solved. Since our main concern is safety, we decide to use the solution satisfying the safe set constraint at the furthest time. A scheme of the Parallel Constraint MPC is depicted in Fig. 1 and a pseudo-code of the controller is available in Alg. 1.

A. Limited Computational Units

If less than N computational cores are available, we should decide on which time steps to try to satisfy the safe set constraint. First of all, we decided to always allocate one computational unit to solve an OCP having the safe-set constraint at time step $r-1$, where r is the time step at

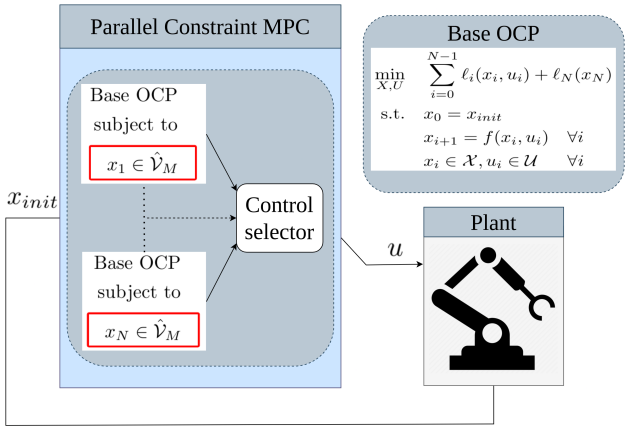

Fig. 1. Parallel MPC scheme. At each control step, N different problems with form (9) are solved in parallel, then the best solution in term of safety is chosen and applied to the plant.

which the safe set constraint was satisfied at the previous MPC loop. This is because, in theory, the safe-set constraint is feasible at that time step.

For allocating the remaining computational units, we have devised three possible strategies to choose the time steps.

- *Uniform*: Approximately equally spaced along the horizon, considering that time step $r-1$ must always be included in the list. For instance, if we had 4 computational units, a horizon $N = 20$, and $r = 9$, we would choose the time steps 4, 8, 14, and 20.
- *High*: The furthest down the horizon (i.e., $N, N-1, N-2, \dots$), which would provide safety for longer.
- *Closest*: The time steps associated to the smallest violations of the safe set constraint at the previous loop.

B. Checking Constraint Satisfaction

Our MPC implementation relies on the Real-Time Iteration (RTI) scheme [25], which boils down to performing a single iteration of Newton's method, without line search. Therefore, after each solver iteration we need to check if constraints are satisfied. In particular, also the system dynamics can be violated. This implies that, even if the predicted state trajectory satisfies all constraints, the real state trajectory obtained by forward integration of the predicted control inputs may violate some constraints. For this reason, for every problem, we need to perform a forward integration of the computed control trajectory, and check the constraints on the resulting state trajectory.

C. Warm Start

As typically done in MPC, we provide an initial guess to warm-start the solver. However, rather than providing a shifted version of the previous solution, we use a different approach, which we have observed to improve the results. As mentioned above, the RTI scheme can result in violation of the dynamics constraints. To help the solver satisfy the dynamics constraints, we provide an initial guess that satisfies them. This guess is obtained by forward integration of the predicted control trajectory.

D. Theoretical Guarantees

From a theoretical standpoint, *Parallel-Constraint* and *Receding-Constraint* MPC share similar properties. Indeed, both ensure *recursive feasibility* under the assumption that $\hat{\mathcal{V}}_M$ is $(N-L)$-step control invariant, and both ensure *safety* using the task-abortion strategy, under Assumptions 1, 2, and 3.

The variants of *Parallel-Constraint* MPC that use a limited number of computational units maintain safety, but they may lose recursive feasibility, depending on how they are implemented. In particular, one way to ensure recursive feasibility is to always try to satisfy the safe-set constraint at the time steps $r-1$ and N. Our implementation of *High* does it, therefore it ensures recursive feasibility. *Uniform* does not necessarily do it, but as long as the ratio between N and the number of computational units K is not large, *Uniform* ensures recursive feasibility under a slightly stronger assumption. Indeed, *Uniform* always tries to satisfy the safe-set constraint at a time step between $\lfloor N - N/K \rfloor$ and N. This ensures recursive feasibility under the assumption that $\hat{\mathcal{V}}_M$ is $\lfloor N-L-N/K \rfloor$-step control invariant. Finally, *Closest* does not necessarily try to satisfy the safe-set constraint at time step N, therefore it does not ensure recursive feasibility.

While sharing the same theoretical properties, our tests show that *Parallel-Constraint* MPC and its computationally cheaper variants in general outperform *Receding-Constraint* MPC. In part this may be due to the use of the RTI scheme [25], which can lead to unfeasible solutions even when the tackled problem is feasible.

V. Simulation Results

This section presents our simulation results[1].

A. Simulation Setup

To evaluate the proposed *Parallel-Constraint* MPC, we have carried out a comparison of many MPC formulations:

- *Naive*: a classic formulation without terminal constraint, i.e., problem (5) with $\mathcal{X}_N = \mathcal{X}$. This is the baseline for all the experiments.
- *Receding*: the formulation (6), using soft constraints for both $x_r \in \hat{\mathcal{V}}_M$ and $x_N \in \hat{\mathcal{V}}_M$. The penalty weight on the receding constraint (10^4) is higher than the terminal one ($w_s = 10^2$) to mimic a hard constraint.
- *Parallel*: a formulation solving N MPC problems in parallel, as described in Alg. 1.
- *Uniform*, *Closest*, *High*: the three formulations of *Parallel-Constraint* MPC using a limited number of computational units (either 4, 8, or 16), as described in Section IV-A.

For the simulations, we have considered a 3-joint manipulator, thus $n_x = 6$, $n_u = 3$. CasADi [26] has been used as symbolic framework for the dynamics, costs, and constraints,

[1]Our open-source code is freely available at `https://github.com/EliasFontanari/safe_mpc/tree/devel`.

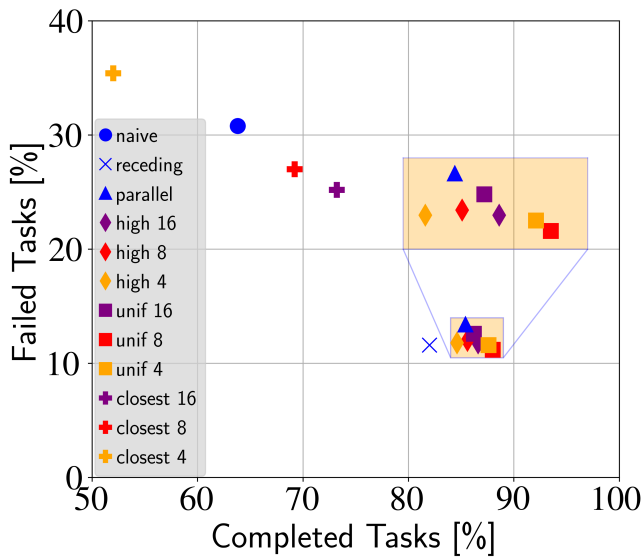

Fig. 2. Number of times each controller completed the task, or violated a constraint (with safety factor $\alpha = 2\%$, whose effect is reported in (12)). The orange rectangle is zoomed in to improve readability.

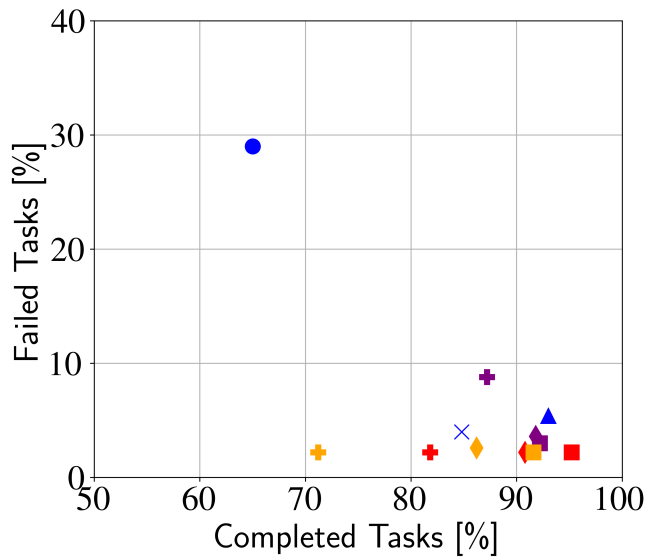

Fig. 3. Number of times each controller completed the task, or violated a constraint (with $\alpha = 10\%$).

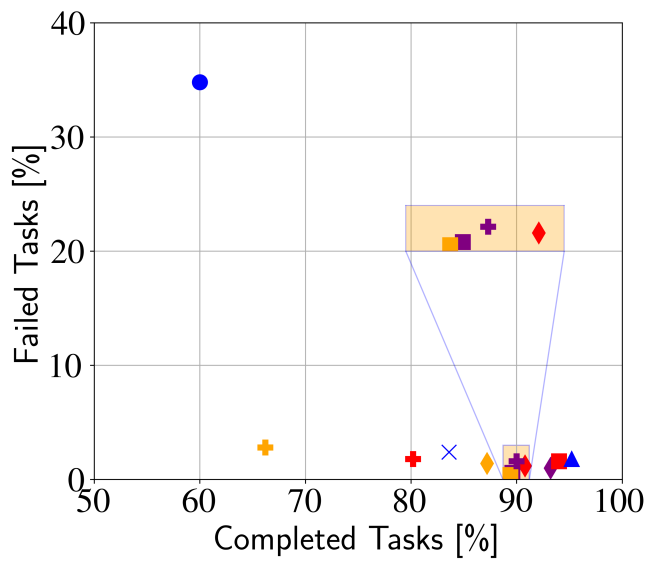

Fig. 4. Number of times each controller completed the task, or violated a constraint (with $\alpha = 15\%$).

while ACADOS [27] as OCPs solver and dynamics' integrator. The task is a setpoint regulation problem with respect to a state purposely chosen near the position limit of the first joint, to test the safety of the controllers:

$$x^{\text{ref}} = (q^{\max} - 0.05, \bar{q}, \bar{q}, 0, 0, 0), \tag{10}$$

with $\bar{q} = (q^{\max} + q^{\min})/2$. We have used as running cost a least-squares function, penalizing deviations from x^{ref} and control efforts:

$$\begin{aligned} l(x, u) &= ||x - x^{\text{ref}}||_Q^2 + ||u||_R^2, \\ Q &= \text{diag}([500, 10^{-4} I_5]), \quad R = 10^{-4} I_3, \end{aligned} \tag{11}$$

where I_h is the identity matrix with size h. Set membership to $\hat{\mathcal{V}}_M$ is verified with the constraint:

$$(1 - \alpha)\phi(x) - ||\dot{q}|| \geq 0, \tag{12}$$

where $\phi(\cdot)$ is a Neural Network (NN) that computes an upper bound on the joint velocity norm [12], and $\alpha \in [0, 1]$ is a safety margin that we introduced to ensure that $\hat{\mathcal{V}}_M \subseteq \mathcal{V}_M$. The PyTorch [28] neural model is integrated inside ACADOS using the L4CASADI [29] library. We have run 500 simulations for each MPC formulation, starting from the same 500 random joint positions q_0 with $\dot{q}_0 = 0$. The time step of the MPC was $dt = 5\,\text{ms}$, and the horizon has been fixed to $N = 35$, so that each iteration takes less than $4\,\text{ms}$ (leaving $1\,\text{ms}$ for further operations, to mimic the timing limitations of a real-time application). Each simulation was stopped when one of these conditions was met: i) the first joint has reached the target, ii) the robot has violated a joint limit, iii) the maximum number of simulation steps (600) has been reached. In the last case, we have tried to trigger the safe abort from the final state, and considered the simulation as a failure if the safe abort has not worked.

Optimistically we have set $L = 0$, assuming that the safe-abort OCP can be solved in under $5\,\text{ms}$. While this is not currently the case, we hope we can achieve this performance in the future, for instance by training a neural network to provide a good warm-start for the solver.

B. Discussion of the results

Fig. 2, 3, and 4 summarize the results for the three tested scenarios, using $\alpha = 2\%, 10\%$, and 15%, respectively. For each value of α we have sampled a different set of initial conditions, making sure that the first MPC problem could be solved for every formulation. The plots show the percentage of tasks that have been completed and failed by each MPC formulation. The percentage of tasks that have been safely

aborted is not directly represented, but it can be retrieved knowing that the three values (completed, failed, aborted) must sum up to 100.

1) Naive, Receding and Parallel: For all values of α, *Naive* has been outperformed by *Receding* and *Parallel*, which have always completed more tasks while failing less tasks. *Parallel* and *Receding* have performed somewhat similarly for $\alpha = 2\%$: only $+4\%$ of completed tasks and $+1.8\%$ of failures for *Parallel*. For higher values of α ($\alpha = 10/15\%$), *Parallel* has outperformed *Receding* in terms of completed tasks ($+8.2/11.6\%$) while failing a similar number of tasks ($+1.4/-0.6\%$). Surprisingly, the performance of *Parallel* has monotonically improved increasing α. This was against our expectation, which was to witness a decrease in both the numbers of failed and completed tasks. In particular, the most significant improvement is in the percentage of failed tasks, which went from 13.4% (for $\alpha = 2\%$) to 1.8% (for $\alpha = 15\%$). This clearly shows that having a sufficiently high value of α is fundamental for getting a high success rate of the safe-abort procedure, which failed 91% of the time for $\alpha = 2\%$, but only 37% of the time for $\alpha = 15\%$.

We have noticed that for $\alpha = 2\%$ the failure rate of the safe abort for *Receding* was significantly lower than for *Parallel* (65% against 91%). To understand why, we have analyzed the distance of the robot joints to the position boundaries at the states used for triggering the safe abort. In particular, we have noticed that the first joint was particularly crucial in this regard, so we have plotted the distance of this joint to its boundary, which can be seen in Fig. 5. We can see how *Parallel* has triggered the safe abort from states where the first joint was (on average) closer to its limit. This can justify why the safe abort has failed more often for *Parallel* than for *Receding*.

We also analyzed the number of time steps taken to reach the target of *Receding* and *Parallel*, considering only the tasks completed by both controllers. *Parallel* was faster to reach the target: on average it needed 225 steps, while *Receding* required 262 steps.

2) Limited Computational Units: Regarding the other versions of *Parallel* using a limited number of computational units, their performance has been mixed. In general, *High* and *Unif* have performed well, approximately matching the performance of *Parallel* when using 16 computational units, and showing a graceful performance degradation with lower numbers of computational units. Even for 4 computational units, these strategies have always outperformed *Receding*, empirically proving that improvements are achievable even with limited computational resources. Surprisingly, for $\alpha = 2\%$ and $\alpha = 10\%$ some variants of *High* and *Unif* outperformed *Parallel*; moreover *Unif* has performed better when using 8 computational units rather than 16 or 4. These unexpected results could be statistical outliers due to the relatively small number of tests carried out (500), or they could be due to some phenomena that we have not yet understood. *Closest* instead has always been the worst performer, especially in terms of completed tasks (often worse than *Receding*).

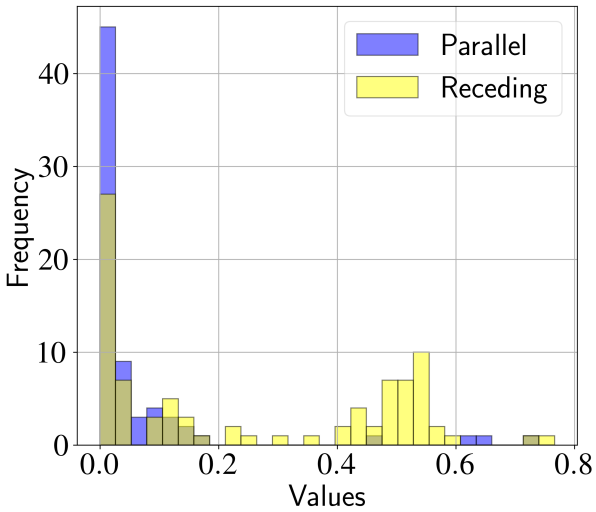

Fig. 5. Distance to the boundary of the first joint when the safe-abort procedure is triggered (with $\alpha = 2\%$).

VI. Conclusions

We have presented *Parallel-Constraint* MPC, a novel MPC formulation that exploits parallel computation to improve safety. Our formulation can be seen as an extension of our previous work on *Receding-Constraint* MPC [21], which also addressed the safety problem, but without exploiting parallel computation. While *Parallel* and *Receding*-Constraint MPC share similar theoretical properties in terms of safety and recursive feasibility, their empirical performance is substantially different. Our results show that exploiting parallel computation can lead to remarkable improvements. For instance, considering their best performance, *Parallel* has completed 13% more tasks than *Receding*, while failing 5% less tasks. Moreover, we have shown that these improvements degrade gracefully when using a limited number of computational cores (4, 8 or 16) and that allocating these units using the right strategy is fundamental.

While the obtained results are highly encouraging, much work is still needed to allow the implementation of these advanced MPC formulations on real hardware. In the future, we would like to bridge the gap that is currently preventing us from validating our work on real robots. In particular, we are currently working on strategies to speed-up the safe-abort procedure, so that it is real-time compatible. Moreover, we are testing our methodology in more challenging scenarios featuring more complex constraints, such as avoiding self collisions and collisions with the environment.

Another aspect that we have not addressed yet is the presence of uncertainties. Exploiting tools from Robust MPC [30], we could generate a nominal state trajectory using our approach (ignoring uncertainties) and then design a local control law to maintain the real system trajectory in a tube centered around the nominal trajectory [31].

REFERENCES

[1] F. Blanchini, "Set invariance in control," *Automatica*, vol. 35, pp. 1747–1767, 1999.

[2] L. Grüne and J. Pannek, *Nonlinear model predictive control*. Springer, 2017.

[3] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs with application to adaptive cruise control," in *IEEE Conference on Decision and Control*, 2014, pp. 6271–6278.

[4] Z. Wu, F. Albalawi, Z. Zhang, J. Zhang, H. Durand, and P. D. Christofides, "Control Lyapunov-Barrier function-based model predictive control of nonlinear systems," *Automatica*, vol. 109, p. 108508, 2019. [Online]. Available: https://doi.org/10.1016/j.automatica.2019.108508

[5] B. Djeridane and J. Lygeros, "Neural approximation of pde solutions: An application to reachability computations," in *IEEE Conference on Decision and Control*, 2006, pp. 3034–3039.

[6] P.-A. Coquelin, S. Martin, and R. Munos, "A dynamic programming approach to viability problems," in *IEEE International Symposium on Approximate Dynamic Programming and Reinforcement Learning*, 2007, pp. 178–184.

[7] F. Jiang, G. Chou, M. Chen, and C. J. Tomlin, "Using neural networks to compute approximate and guaranteed feasible hamilton-jacobi-bellman pde solutions," 2016. [Online]. Available: https://www.arxiv.org/abs/1611.03158

[8] V. Rubies-Royo and C. Tomlin, "Recursive Regression with Neural Networks: Approximating the HJI PDE Solution," in *International Conference on Learning Representations*, 2017.

[9] K. C. Hsu, V. Rubies-Royo, C. J. Tomlin, and J. F. Fisac, "Safety and Liveness Guarantees through Reach-Avoid Reinforcement Learning," *Robotics: Science and Systems*, 2021.

[10] C. Dawson, S. Gao, and C. Fan, "Safe Control With Learned Certificates : A Survey of Neural Lyapunov , Barrier , and Contraction Methods for Robotics and Control," *IEEE Transactions on Robotics*, vol. 39, no. 3, pp. 1749–1767, 2023.

[11] Y. Zhou, D. Li, Y. Xi, and Y. Xu, "Data-driven approximation for feasible regions in nonlinear model predictive control," 2020. [Online]. Available: https://arxiv.org/abs/2012.03428

[12] A. La Rocca, M. Saveriano, and A. Del Prete, "VBOC: Learning the Viability Boundary of a Robot Manipulator using Optimal Control," *IEEE Robotics and Automation Letters*, 2023.

[13] S. Li and O. Bastani, "Robust Model Predictive Shielding for Safe Reinforcement Learning with Stochastic Dynamics," in *IEEE International Conference on Robotics and Automation*, 2020, pp. 7166–7172.

[14] O. Bastani, S. Li, and A. Xu, "Safe Reinforcement Learning via Statistical Model Predictive Shielding," in *Robotics Science and Systems (RSS)*, 2021.

[15] K. M. Abughalieh and S. G. Alawneh, "A survey of parallel implementations for model predictive control," *IEEE Access*, vol. 7, pp. 34 348–34 360, 2019.

[16] E. Adabag, M. Atal, W. Gerard, and B. Plancher, "MPCGPU: Real-Time Nonlinear Model Predictive Control through Preconditioned Conjugate Gradient on the GPU," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024. [Online]. Available: http://arxiv.org/abs/2309.08079

[17] S. H. Jeon, S. Hong, H. J. Lee, C. Khazoom, and S. Kim, "Cusadi: A gpu parallelization framework for symbolic expressions and optimal control," *arXiv preprint arXiv:2408.09662*, 2024.

[18] J. Rajamaki and P. Hamalainen, "Sampled Differential Dynamic Programming," in *American Control Conference (ACC)*, 2016, pp. 2182–2189.

[19] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.

[20] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, "Predictive sampling: Real-time behaviour synthesis with mujoco," *arXiv preprint arXiv:2212.00541*, 2022.

[21] G. Lunardi, A. L. Rocca, M. Saveriano, and A. D. Prete, "Receding-Constraint Model Predictive Control using a Learned Approximate Control-Invariant Set," in *IEEE International Conference on Robotics and Automation*, 2024.

[22] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[23] E. C. Kerrigan and J. M. Maciejowski, "Soft constraints and exact penalty functions in model predictive control," in *UKACC International Conference (Control)*, 2000.

[24] M. N. Zeilinger, M. Morari, and C. N. Jones, "Soft constrained model predictive control with robust stability guarantees," *IEEE Transactions on Automatic Control*, vol. 59, no. 5, pp. 1190–1202, 2014.

[25] M. Diehl, R. Findeisen, F. Allgöwer, H. G. Bock, and J. P. Schlöder, "Nominal stability of real-time iteration scheme for nonlinear model predictive control," *IEE Proceedings-Control Theory and Applications*, vol. 152, no. 3, pp. 296–308, 2005.

[26] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[27] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "Acados: a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, pp. 147–183, 2019.

[28] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[29] T. Salzmann, J. Arrizabalaga, J. Andersson, M. Pavone, and M. Ryll, "Learning for casadi: Data-driven models in numerical optimization," in *6th Annual Learning for Dynamics & Control Conference*. PMLR, 2024, pp. 541–553.

[30] S. Yu, C. Maier, H. Chen, and F. Allgöwer, "Tube MPC scheme based on robust control invariant set with application to Lipschitz nonlinear systems," *Systems and Control Letters*, vol. 62, no. 2, pp. 194–200, 2013.

[31] D. Q. Mayne, E. C. Kerrigan, E. J. Van Wyk, and P. Falugi, "Tube-based robust nonlinear model predictive control," *International Journal of Robust and Nonlinear Control*, vol. 21, no. 11, pp. 1341–1353, 2011.